

数据同步

对视频表，用户表进行 mysql 数据同步到 elasticsearch 的视频、用户索引（es 索引和 mysql 表类似，索引下的文档和 mysql 的每一行类似）

对视频表 and 用户表数据有影响的操作都会同步到视频索引中

```
UploadAndEditController.java  UploadAndEditServiceImpl.java
25  @RestController  Amumu
26  @RequestMapping("/createCenter")
27  @Api(tags = "上传视频和编辑视频")
28  @CrossOrigin(value = "*")
29  @Slf4j
30  public class UploadAndEditController {
31      @Resource 5个用法
32      UploadAndEditService uploadAndEditService;
33
34      @ApiOperation("上传视频") Amumu
35      @PostMapping("/uploadTotal")
36      public Result<Boolean> uploadTotal(@RequestBody UploadVideoRequest uploadVideoRequest) throws IOException {
37          log.info("上传视频");
38          return uploadAndEditService.uploadTotal(uploadVideoRequest);
39      }
40      @ApiOperation("上传分片并获取视频第一帧图片和合并后的路径") Amumu
41      @PostMapping("/uploadPart")
42      public Result<List<String>> uploadPart(@ModelAttribute UploadPartRequest uploadPartRequest) throws EncoderException,
43          IOException, ServerException, InsufficientDataException, ErrorResponseException, NoSuchAlgorithmException,
44          InvalidKeyException, InvalidResponseException, XmlParserException, InternalException {
45          return uploadAndEditService.uploadPart(uploadPartRequest);
46      }
47      @ApiOperation("编辑视频") Amumu
48      @PostMapping("/edit")
49      public Result<Boolean> edit(@ModelAttribute EditVideoRequest editVideoRequest){
50          log.info("1");
51          return uploadAndEditService.edit(editVideoRequest);
52      }
53      @ApiOperation("删除视频") Amumu
54      @PostMapping("/delete")
55      public Result<Boolean> delete(@RequestBody DeleteVideoRequest deleteVideoRequest){
56          log.info("1");
57          return uploadAndEditService.delete(deleteVideoRequest);
58      }
59  }
```

将视频实体转换成 map 对象再给 map 添加操作的表名和是新增修改删除中的哪种操作类型做区分，然后调用 openfeign 远程接口生成数据同步消息

```
//将新增的视频信息转换成map
Map<String, Object> map = objectMapper.convertValue(video, Map.class);
//指定表名是视频表
map.put(TABLE_NAME, VIDEO_TABLE_NAME);
//指定操作类型是新增
map.put(OPERATION_TYPE, OPERATION_TYPE_ADD);
//发送数据同步消息
client.sendDBChangeNotice(map);
```

```

//将修改后的视频信息转换成map
Map<String, Object> map = objectMapper.convertValue(video, Map.class);
//指定表名是视频表
map.put(TABLE_NAME, VIDEO_TABLE_NAME);
//指定操作类型是修改
map.put(OPERATION_TYPE, OPERATION_TYPE_UPDATE);
//发送数据同步消息
client.sendDBChangeNotice(map);

```

```

//将要被删除的视频信息转换成map, 该map只有id有价值
Map<String, Object> map = objectMapper.convertValue(video, Map.class);
//指定表名是视频表
map.put(TABLE_NAME, VIDEO_TABLE_NAME);
//指定操作类型是删除
map.put(OPERATION_TYPE, OPERATION_TYPE_DELETE);
//发送数据同步消息
client.sendDBChangeNotice(map);

```

用户表则是在账号密码注册、手机号邮箱号登录（新用户采用手机号邮箱号登录时如果对应手机号邮箱号在数据库中不存在会注册新账号）、修改用户信息时会发送用户表数据同步消息

```

@Resource 1个用法
RegisterService registerService;
@ApiOperation("密码方式注册")  Amumu
@PostMapping("/passwordRegister")
public Result<Integer> passwordRegister(@RequestBody PasswordRegisterRequest passwordRegisterR
    return registerService.passwordRegister(passwordRegisterRequest);
}
}

```

```

@ApiOperation("手机号登录/注册")  Amumu
@PostMapping(🌐"/phoneNumberLogin")
public Result<LoginResponse> phoneNumberLogin(@RequestBody PhoneNumberLoginRequest phoneNumberLoginRequest,
ServerWebExchange exchange) {
    Map<String, Object> map = loginService.phoneNumberLogin(phoneNumberLoginRequest);
    if (map != null) {
        Integer userId=(Integer) map.get(USER_IDENTITY);
        String shortJwt = (String) map.get(SHORT_TOKEN);
        String longJwt=(String)map.get(LONG_TOKEN);
        exchange.getResponse().getHeaders().add(SHORT_TOKEN, shortJwt);
        ResponseCookie cookie = ResponseCookie.from(LONG_TOKEN, longJwt)
            .httpOnly(true)
            .path("/")
            .maxAge( maxAgeSeconds: 60 * 30)
            .build();
        exchange.getResponse().addCookie(cookie);
        log.info("手机号登录的id"+userId);
        return Result.data(new LoginResponse().setStatus(true).setUserId(userId));
    }
    return Result.data(new LoginResponse().setStatus(false));
}

@ApiOperation("邮箱号登录/注册")  Amumu
@PostMapping(🌐"/mailLogin")
public Result<LoginResponse> mailLogin(@RequestBody MailLoginRequest mailLoginRequest, ServerWebExchange exchange) {
    Map<String, Object> map = loginService.mailLogin(mailLoginRequest);
    if (map != null) {

```

```

@PostMapping(🌐"/editUserInfo")  Amumu
@ApiOperation("编辑个人信息")
public Result<Boolean> editUserInfo(@RequestParam("file") Multi
userId,@RequestParam("nickname") String nickname,@RequestParam
InsufficientDataException, ErrorResponseException, IOException
InvalidResponseException, XmlParserException, InternalException
    log.info("1");
    return userInfoService.editSelfInfo(file,userId,nickname,in
}

```

同步消息生成步骤与视频数据同步消息生成步骤类似

```

//将新创建的用户的信息转换成map
Map<String, Object> userMap = objectMapper.convertValue(newUser, Map.class);
//指定表名是用户表
userMap.put(TABLE_NAME, USER_TABLE_NAME);
//指定操作类型是新增、删除、修改中的新增
userMap.put(OPERATION_TYPE, OPERATION_TYPE_ADD);
client.sendDBChangeNotice(userMap);

```

```

ObjectMapper objectMapper = new ObjectMapper();
//将修改后的用户的信息转换成map
Map<String, Object> userMap = objectMapper.convertValue(newUser, Map.class);
//指定表名是用户表
userMap.put(TABLE_NAME, USER_TABLE_NAME);
//指定操作类型是新增、删除、修改中的修改
userMap.put(OPERATION_TYPE, OPERATION_TYPE_UPDATE);
client.sendDBChangeNotice(userMap);

```

```

//将删除的用户的信息转换成map，该map中只有用户的id是有值的，也只需要用户的id有值
// 因为es删除一个文档只需要该文档的id即可，也即map中的用户id
Map<String, Object> userMap = objectMapper.convertValue(newUser, Map.class);
//指定表名是用户表
userMap.put(TABLE_NAME, USER_TABLE_NAME);
//指定操作类型是新增、删除、修改中的删除
userMap.put(OPERATION_TYPE, OPERATION_TYPE_DELETE);
client.sendDBChangeNotice(userMap);

```

消息生成后在消息队列中进行消费

将视频表的增删改和用户表的增删改区分开，存6个redis的list数据类型键值对

@Resource 5 个用法

RedisTemplate objectRedisTemplate;

/**

*根据map的键值对不同将其存入特定redis键值对

*/

@Override 0 个用法 Amumu

public void onMessage(MessageExt messageExt){

String json = new String(messageExt.getBody(), StandardCharsets.UTF_8);

HashMap<String, Object> map ;

try {

//将json消息转成map

map = objectMapper.readValue(json, HashMap.class);

} catch (JsonProcessingException e) {

throw new RuntimeException(e);

}

//如果操作类型是新增的话

if(map.get(OPERATION_TYPE).equals(OPERATION_TYPE_ADD)){

log.info("add");

map.remove(OPERATION_TYPE);

//如果表是视频表的话

if(map.get(TABLE_NAME).equals(VIDEO_TABLE_NAME)){

log.info("video");

map.remove(TABLE_NAME);

objectRedisTemplate.opsForList().rightPush(VIDEO_ADD_KEY, map);

}else {

map.remove(TABLE_NAME);

objectRedisTemplate.opsForList().rightPush(USER_ADD_KEY, map);

}

}

else if(map.get(OPERATION_TYPE).equals(OPERATION_TYPE_DELETE)){

log.info("delete");

map.remove(OPERATION_TYPE);

map.remove(TABLE_NAME);

objectRedisTemplate.opsForList().rightPush(VIDEO_DELETE_KEY, map);

}

else{

log.info("update");

map.remove(OPERATION_TYPE);

if(map.get(TABLE_NAME).equals(VIDEO_TABLE_NAME)){

log.info("video");

map.remove(OPERATION_TYPE);

objectRedisTemplate.opsForList().rightPush(VIDEO_UPDATE_KEY, map);

}

在 es 中执行每 15 分钟一次的 xxl-job 配置的定时任务,先查有没有全量同步过,没有就进行全量同步并将同步标识设置为 true

```
/**
 *同步MySQL数据到ES
 */
@XxlJob("mysqlToEs") 1个用法 Amumu
public void mysqlToEsHandler() throws Exception {
    //查询是否已同步过视频数据,没有的话全量同步视频数据
    if (hasSynchronousVideo == false) {
        mysqlToEsService.videoMysqlToEs();
        mysqlToEsService.updateVideoData();
        hasSynchronousVideo = true;
    }
    //查询是否已同步过用户数据,没有的话全量同步用户数据
    if (hasSynchronousUser == false) {
        mysqlToEsService.userMysqlToEs();
        mysqlToEsService.updateUserData();
        hasSynchronousUser = true;
    }
}
```

全量同步是查出所有的 mysql 视频、用户表数据并转换成 map 作为文档插入 es 中

然后进行增量同步,先将 redis 中六个 list 数据类型的键值对取出来,再按先增加删除同步再修改同步的顺序进行数据同步

```

//将redis存储的视频、用户相关增删改的操作记录下来进行增量同步
List<HashMap<String, Object>> videoAddList = objectRedisTemplate.opsForList().range(Constant.VIDEO_INDEX_NAME, start: 0, end: -1);
List<HashMap<String, Object>> videoDeleteList = objectRedisTemplate.opsForList().range(Constant.VIDEO_INDEX_NAME, start: 0, end: -1);
List<HashMap<String, Object>> videoUpdateList = objectRedisTemplate.opsForList().range(Constant.VIDEO_INDEX_NAME, start: 0, end: -1);
List<HashMap<String, Object>> userAddList = objectRedisTemplate.opsForList().range(Constant.USER_INDEX_NAME, start: 0, end: -1);
List<HashMap<String, Object>> userUpdateList = objectRedisTemplate.opsForList().range(Constant.USER_INDEX_NAME, start: 0, end: -1);
//先执行增删同步再进行修改同步，防止修改时该文档已删除或者还未添加
if (videoAddList.size() > 0) {
    mysqlAddToEs(Constant.OPERATION_ADD, videoAddList, Constant.VIDEO_INDEX_NAME);
}
if (videoDeleteList.size() > 0) {
    mysqlAddToEs(Constant.OPERATION_DELETE, videoDeleteList, Constant.VIDEO_INDEX_NAME);
}

if (userAddList.size() > 0) {
    mysqlAddToEs(Constant.OPERATION_ADD, userAddList, Constant.USER_INDEX_NAME);
}

```

同步时根据表的不同和操作类型的不同创建不同的 es 批量操作对象，再遍历新增、删除、修改的集合，将每一个单独的新增、删除、修改操作对象添加到批量操作对象中，

再递归的对 es 进行批量操作，收集失败的单次操作重新添加到新的批量操作对象中，进行一轮新的批量操作直到失败请求数为 0 或递归次数大于 10（最大程度减少失败请求次数同时设定次数为 10 防止堆栈溢出）

增删操作执行完后将当前索引的所有文档 id 存储到 HashSet 中，添加单独修改对象到批量操作对象中前先查询 HashSet 中是否有该单独修改对象对应的文档 id，没有的话就说明该文档的最终状态是被删除，则该单独修改对象就不会被添加到批量操作对象中

方案二则是先新增再修改再删除，这样不用引入 HashSet，但增加了 es 的负担，面试时可把两个方案都说出来